

Dokumentation Barrista88 GUI - Projekt

Anfängliche Idee und erstes Grundgerüst

Zuerst habe ich mir überlegt wie das Projekt optisch aussehen soll. Ich habe mich dafür entschieden, ein Bild einer Kaffeemaschine welche über ein Display besitzt als Grundlage in meinem HTML - Dokument zu nutzen und auf dieses die Funktionalität mit JavaScript zu legen.

Das Bild ist KI- generiert (ChatGPT):



Als nächstes habe ich ein HTML Dokument erstellt, mit einem DIV als Container für das Bild und entsprechend die Machine:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link rel="stylesheet" href="../css/styles.css">
  <script src="../js/barrista88.js" defer></script>
  <title>Barrista88</title>
</head>
<body>
  <div class="maschinencontainer">
```

```
        
    </div>

</body>
</html>
```

Wie man dem Code entnehmen kann, habe ich mich dazu entschlossen, HTML, CSS und JavaScript Code in verschiedenen Dokumenten zu speichern. So wird das Projekt für mich übersichtlicher und lässt sich leichter umgestalten und erweitern.

Erste Schritte um Funktionalität aufzubauen.

Um die erste Funktionalität herzustellen, habe ich zwei weitere DIVs in meinem HTML - Dokument erstellt.

Eines, welches als Display/zur Ausgabe meines Codes in dem Dokument dienen soll und eines, welches den An/Aus Schalter der Maschine repräsentiert. Diese beiden DIVs habe ich in das bereits bestehende DIV untergeordnet und mit entsprechenden IDs versehen:

```
<div class="maschinencontainer">

    <div id="display">
    </div>

    <div id="power"></div>

</div>
```

Anschließend habe ich meinen HTML Elementen CSS Attribute zugewiesen.

```
.maschinencontainer{
    position:relative;
    width:100vw;
    max-width: 100%;
    height:100vh;
    max-height:100%;
    margin:0 auto;
    aspect-ratio: 3 / 2;
}

.kaffeemaschine{
    display:block;
    width: 100%;
    height: 100%;
}

#display{
    border: 2px solid red;
```

```
    position: absolute;
    top: 16.75%;
    left: 32.5%;
    width: 35%;
    height: 33%;
    background-color: black;
    display: flex;
    justify-content: center;
    align-items: center;
}

#power{
    border: 2px solid red;
    position: absolute;
    top: 57%;
    left: 69%;
    width: 3.75%;
    height: 6%;
    cursor: pointer;
}
```

Das Bild füllt so den ganzen Bildschirm, der Container der Maschine passt sich dem jeweiligen Bildschirm an und durch die absolute Positionierung der anderen DIVs kann ich diese nun genau so positionieren, wie ich es gerne möchte, ohne dass diese die Position verändern, wenn sich die Bildschirmverhältnisse ändern. Ich habe bewusst rote Umrandungen für die DIVs aktiviert, um sehen zu können, wo und wie diese in einem Browser dargestellt werden. Dadurch konnte ich dann die passenden Werte für die Positionierung (top & left) sowie für die Größe (width & height) ermitteln.



Abbildung: DIVs positionieren und anpassen



Abbildung: DIVs sind korrekt positioniert und angepasst

Als nächstes habe ich in JavaScript eine Variable mit booleschem Wert erstellt, welche den An/Aus Zustand der Maschine darstellen soll:

```
let power = false;
```

Der Standardzustand ist "false", die Maschine ist aus. Dann habe ich 2 Konstanten erstellt, eine für das Display, eine für den Power-Knopf und diese mit den jeweiligen DIVs verknüpft um sie in JavaScript ansprechen zu können.

```
const displayElement = document.getElementById("display");  
const powerButton = document.getElementById("power");
```

Dann habe ich einen Event Listener mit dem Powerknopf verknüpft, der bei einem Klick den Wert der Variable "power" umdreht. Aus "false" wird "true" und umgekehrt -> aus An wird Aus. Dazu eine "if" Abfrage die den Power Status checkt und eine entsprechende Meldung im Display ausgibt

```
powerButton.addEventListener ("click", function (){  
    power = !power;  
    anzeige();  
})  
  
function anzeige(){  
    if(power){  
        displayElement.innerText = "Die Maschine ist an.";  
    }  
    else{  
        displayElement.innerText = "Die Maschine ist aus.";  
    }  
}  
  
anzeige();
```

Mithilfe einer CSS-Media-Query fange ich das Hochformat ab und drehe den Inhalt um 90 Grad, um eine querformatige Darstellung zu simulieren. Dadurch wird der Endnutzer dazu bewegt, das Gerät ins Querformat zu drehen oder die Anwendung auf einem geeigneten Gerät zu nutzen



Abbildung: Erste Funktionalität und Ausgabe im HTML Dokument

Erste graphische Darstellung von Zuständen und Menüs

Nun da die erste Funktionalität hergestellt war, wollte ich das ganze natürlich ausarbeiten und verfeinern. Einfache Textausgaben sind doch etwas langweilig, ich wollte schicke Menüs, die optisch etwas hermachen und interaktiv sind.

Meine erste Idee, dies umzusetzen, war, die jeweiligen Menüs in einer anderen HTML Datei zu coden und dann diese Datei in das Display zu laden. Das erscheint mir sinnvoll und elegant, jede Seite die auf dem Display angezeigt wird, ist eine eigene Seite mit eigenem Quellcode auf dem Datenträger.

Ich recherchierte also, wie das umsetzbar wäre, baute meine `hauptmenü.html` Seite und habe diese mit folgendem Code in das Display geladen:

```
async function hauptMenü() {  
  const response = await fetch('hauptmenü.html');  
  const html = await response.text();  
  displayElement.innerHTML = html;  
}
```

Die Funktion hat sich die html Datei in eine Konstante geladen (fetch), diese musste aber erst in Text konvertiert werden, um korrekt im Display angezeigt zu werden. Dies erfolgt in dem Schritt "`const html = await response.text()`". Zuguterletzt wurde der html-code in Textform an das Display DIV übergeben, welches das Hauptmenü dann entsprechend gerendert hat.

Das entsprechende HTML Dokument war aber eher leer, also hatte ich mich dazu entschieden, das entsprechende CSS und ggfs. JS direkt in dem Dokument zu schreiben. Das war inkonsistent zu meinem ursprünglichen Plan, die jeweiligen Sprachen voneinander zu trennen und mein Code insgesamt war so auch stärker fragmentiert, was Änderungen erschwerte. Also habe ich ein wenig Recherche zur "Best Practice" für so

ein Projekt betrieben und gelernt, dass man idealerweise alle html Dokumente in diesem Fall in einem einzigen HTML Dokument unterbringt. Ich habe also ein weiteres DIV in meinem Display-Container erstellt, welches nun das Hauptmenü enthielt (unterteilt in weitere DIVs, je eines pro Untermenü)

```
<div id="display">

    <div id="hauptmenü">
        <button class="grid-button">
            
            Getränke</button>
        <button class="grid-button">
            
            Auffüllen</button>
        <button class="grid-button">
            
            Reinigen</button>
        <button class="grid-button">
            
            Statistik</button>
        <button class="grid-button">
            
            ?</button>
        <button class="grid-button">
            
            Spezial</button>
    </div>

</div>
```

Dazu kam ein Bild, welches den "Aus"-Zustand darstellen sollte.

In CSS habe ich das ganze entsprechend meinen Vorstellungen gestylt:

```
#hauptmenü {
    display: grid;
    grid-template-columns: repeat(2, 1fr);
    grid-template-rows: repeat(3, 1fr);
    gap: 15px;
    box-sizing: border-box;
    width: 100%;
```

```
height: 100%;
padding: 20px;
background-color: black;
}

.grid-button {
background-color: transparent;
color: white;
border: 2px solid white;
border-radius: 12px;
font-size: 16px;
cursor: pointer;
letter-spacing: 1px;

display: flex;
flex-direction: column;
align-items: center;
justify-content: center;
gap: 10px;

transition: all 0.2s ease;
}

.grid-button:hover {
background-color: rgba(255, 255, 255, 0.1);
transform: scale(1.02);
}

.grid-button img {
max-height: 40px;
width: auto;
display: block;
}
```

Ich habe mich für ein Grid - Layout als Grundsatz entschieden, um die einzelnen Buttons entsprechend eines Rasters anordnen zu können. Dieses Raster habe ich von einer KI generieren lassen. Die einzelnen Elemente in dem raster sind flex - Elemente, sodass sie ihre Größe und Position dynamisch anpassen können. Zu jedem UNtermenü habe ich ein entsprechendes Bild gesucht, welches dieses räsentiert und in die HTML-Datei eingebunden. Den Abschluss machen ein paar nette Effekte, die die jeweiligen Menüs verändern, sobald man mit der Maus darüberfährt.

```
.grid-button:hover {
background-color: rgba(255, 255, 255, 0.1);
transform: scale(1.02);
}
```

Die Hintergrundfarbe verändert sich, sowie die Größe. Dadurch ist dem Nutzer schnell ersichtlich, welche Funktion er auswählt. Auch habve ich den Mauscursor verändert, sobald er über die einzelnen Menüs und den PowerButton schwebt:

```
cursor: pointer;
```



Abbildung: Mauscursor

Dadurch soll dem Nutzer gezeigt werden, welche Flächen interaktiv sind und welche nicht.

So wie es jetzt ist, wird nun aber alles gleichzeitig im Display angezeigt, was ja nicht sein soll. Die Lösung hierfür ist, eine CSS Klasse zu erstellen, welche Inhalte ausblendet, und diese dann mit JavaScript an die Elemente zu vergeben, welche je nach Zustand nicht angezeigt werden sollen:

```
.versteckt{  
  display: none !important;  
}
```

```
const ausDisplay = document.getElementById("aus");  
const anDisplay = document.getElementById("hauptmenü");  
  
    async function anAus(){      //An-Aus Anzeige  
  
        if(power){  
            ausDisplay.classList.add("versteckt");  
            hauptMenü();  
        }  
        else if(!power){  
            ausDisplay.classList.remove("versteckt");  
            anDisplay.classList.add("versteckt");  
        }  
    }
```

2 Konstanten, die sich wieder die entsprechenden Elemente aus dem html Dokument holen (getElementById). Beim Ausführen der abgebildeten Funktion wird die Klasse dem DOM-Element (das in der Konstante

gespeichert ist) zugewiesen, um diese ggfs auszublenden.

Das fertige Resultat sieht dann so aus:



Abbildung: Maschine ist aus



Abbildung: Die Maschine ist An und zeigt das Hauptmenü

Erste Tests und Fehlerbehebung

Ich habe die Webseite zu Testzwecken auf meinem Mobiltelefon geöffnet. Die Darstellung im Hochformat war im Vergleich zum Browser verzerrt:



Über eine Media Query in CSS fange ich das Hochformat ab und drehe den Inhalt um 90 Grad, um eine querformatige Darstellung zu simulieren und den Endnutzer so dazu zu bewegen, das Gerät zu wenden oder gleich ein anderes zu nutzen.

```
@media all and (orientation: portrait) {  
  body {  
    transform: rotate(90deg);  
    transform-origin: center center;  
    width: 100vh;  
    height: 100vw;  
    position: absolute;  
    top: 50%;  
    left: 50%;  
    translate: -50% -50%;  
  }  
}
```

```
}  
}
```

In dem vorherigen Bild ist auch zu erkennen, dass einige der Auswahlelemente nicht in das Display passten und darüber hinaus ragten. Dies war auch in einem späteren Menü im Querformat der Fall:



Abbildung: Elemente ragen übers Display hinaus

Mithilfe von KI bin ich dann zu folgender Lösung gekommen:

```
#spezialmenü {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);
```

```
    grid-template-rows: repeat(2, 1fr);  
}
```

Das Menü ist nun 3 Spalten breit und 2 Reihen hoch statt andersherum und:

```
.grid-button {  
    overflow: hidden;  
    min-height: 0;  
}
```

Die einzelnen Elemente haben nun keine Minimalgröße mehr und durch `overflow:hidden;` wird beim Verändern der Größe in den Elementen alles was übersteht abgeschnitten und dadurch verhindert, dass diese über das Elternelement hinausragen.



_Abbildung: Das Menü ist jetzt neu angeordnet und steht nicht über.



Abbildung: Auch auf einem kleineren Display im Querformat steht nichts mehr über

Nun sind auf einem kleineren Display die Elemente nicht klar zu identifizieren, es zeigt aber auch klar an, dass diese Anwendung nicht für ein entsprechendes Gerät geeignet ist. Das Display selbst verschiebt sich zwar, das steht auf meiner Prioritätsliste aber sehr weit unten im Moment.

Letzte Schritte zu voller Funktionsfähigkeit

Ich habe die Untermenüs in denen eine Auswahl getroffen werden soll ebenso aufgebaut, wie das Hauptmenü und mit passenden Bildern versehen.

ich habe eine Variable angelegt, welche die Nutzerwahl speichert

```
let auswahl;
```

Dazu kamen verschiedenen Konstanten für die einzelnen Elemente

```
const powerButton = document.getElementById("power");
const ausDisplay = document.getElementById("aus");
const anDisplay = document.getElementById("hauptmenü");
const getränkeMenü = document.getElementById("getränkemenü");
const spezialMenü = document.getElementById("spezialmenü");
```

Dazu jeweils einen Event Listener, welcher auf einen Klick reagiert, und die entsprechende Auswahl in die Variable einträgt

```
//Hauptmenü-knöpfe
btnGetränke.addEventListener("click", function(){
  if(!power) return;
  auswahl="getränke";
  zeigeMenü();
});
```

```
btnAuffüllen.addEventListener("click", function(){
    if(!power) return;
    auswahl="auffüllen";
});

btnReinigen.addEventListener("click", function(){
    if(!power) return;
    auswahl="reinigen";
});

btnStatistik.addEventListener("click", function(){
    if(!power) return;
    auswahl="statistik";
});

btnGeheim.addEventListener("click", function(){
    if(!power) return;
    auswahl="geheim";
});

btnSpezial.addEventListener("click", function(){
    if(!power) return;
    auswahl="spezial";
    zeigeMenü();
});
```

Sowie eine Funktion, die dann aufgerufen wird, welche dann, je nach Wahl, die entsprechenden Klassen entweder versteckt oder sichtbar macht:

```
function zeigeMenü(){
    anDisplay.classList.add("versteckt");

    switch(auswahl){
        case "getränke":
            anDisplay.classList.add("versteckt");
            getränkeMenü.classList.remove("versteckt");
            btnAbbruch.classList.remove("versteckt");
            break;
        case "reinigen":
            anDisplay.classList.add("versteckt");

            btnAbbruch.classList.remove("versteckt");
            break;
        case "statistik":
            anDisplay.classList.add("versteckt");

            btnAbbruch.classList.remove("versteckt");
            break;
        case "auffüllen":
            anDisplay.classList.add("versteckt");
```

```

        btnAbbruch.classList.remove("versteckt");
        break;
    case "geheim":
        anDisplay.classList.add("versteckt");

        btnAbbruch.classList.remove("versteckt");
        break;
    case "spezial":
        anDisplay.classList.add("versteckt");
        spezialMenü.classList.remove("versteckt");
        btnAbbruch.classList.remove("versteckt");
        break;
    }
}

```

Nach diesen Prinzipien habe ich alle Menüs aufgebaut und nach und nach Funktionen aus dem vorherigen Projekt übernommen und etwas abgeändert um die Funktion mit der neuen Anwendung zu ermöglichen:

```

function betriebsstoffePrüfen(auswahl){
    switch(auswahl){
        case "Kaffee":
            if(bohnen >=10 && wasser >=0.3) return true;
            else return false;
            break;
        case "Brühe":
            if(brühe >= 10 && wasser >= 0.3) return true;
            else return false;
            break;
        case "Kakao":
            if(milch >= 0.3 && kakao >= 10) return true;
            else return false;
        case "Milch":
            if(milch >= 0.3) return true;
            else return false;
    }
}

```

Beispiel: Funktion zum Prüfen der Betriebsstoffe, jetzt angepasst auf die neue Anwendung

Stolpersteine beim Geheimmenü

Um das Geheimmenü zu implementieren, habe ich ein Formular in meinem HTML Dokument erstellt:

```

<form class="versteckt"
      id="geheimmenü"
      onsubmit="return false;">
  <fieldset>
  <legend>Frage 1</legend>

```

```
<label for="frage1">Welches ist die Höchstgeschwindigkeit einer
unbeladenen Schwalbe?</label>
<input type="text"
      id="frage1">
</fieldset>

<fieldset>
<legend>Frage 2</legend>
<label for="frage2">Die Antwort auf die ultimative Frage nach dem
Leben, dem Universum und dem ganzen rest: </label>
<input type="text"
      id="frage2">
</fieldset>
<button type="button"
      id="btnGeheimSenden"
      class="grid-button">Antworten überprüfen</button>

</form>
```

Dazu entsprechende Variablen in JavaScript erstellt, sowie einen EventListener mit Funktion, welche die Eingaben prüft:

```
const btnGeheimSenden = document.getElementById("btnGeheimSenden");
const frage1Input = document.getElementById("frage1");
const frage2Input = document.getElementById("frage2");

let geheimnis = false;

//Geheimmenü-Check

btnGeheimSenden.addEventListener("click", function(){
    const geheim1 = frage1Input.value.toLowerCase();
    const geheim2 = frage2Input.value.toLowerCase();

    if(geheim1.includes("afrikanische") &&
    geheim1.includes("europäische") && geheim1.includes("schwalbe") &&
    geheim2.includes("42")){
        geheimnis = true;
        alert("Das Spezialmenü wurde freigeschaltet");

        frage1Input.value = "";
        frage2Input.value = "";
        geheimMenü.classList.add("versteckt");
        hauptMenü();
    }
    else{
        fehler +=1;
        alert("Die Eingabe ist falsch")
    }

    if(fehler === 3){
```

```
gesperrt!");  
  
        alert("WARNUNG: Zu viele Fehler, Menü wird  
        btnGeheim.classList.add("versteckt");  
        geheimMenü.classList.add("versteckt");  
        hauptMenü();  
    }  
});
```

Hierbei handelt es sich um den voll funktionsfähigen und mehrfach überarbeiteten Code. Mein erster Entwurf beinhaltete einige kleine Fehler und war funktionslos. Rechtschreibfehler und falsche Variablen, statt **frage1Input.value.toLowerCase();** habe ich ursprünglich nur **frage1Input.toLowerCase();** verwendet, wodurch die Eingabe nicht korrekt abgefragt und umgewandelt wurde. Um meinen Code zu korrigieren und die Fehler zu finden, habe ich KI als debugging Hilfe genutzt. Die KI gab mir den Hinweis, mein Formular um das Attribut **onsubmit="return false;** zu erweitern, um so ein erneutes Laden der Seite und somit einen Abbruch des Skriptes zu verhindern.

Das fertige Menü ist nun voll funktionsfähig, optisch bin ich aber noch nicht 100% zufrieden, die Feinschliffphase folgt am Ende.

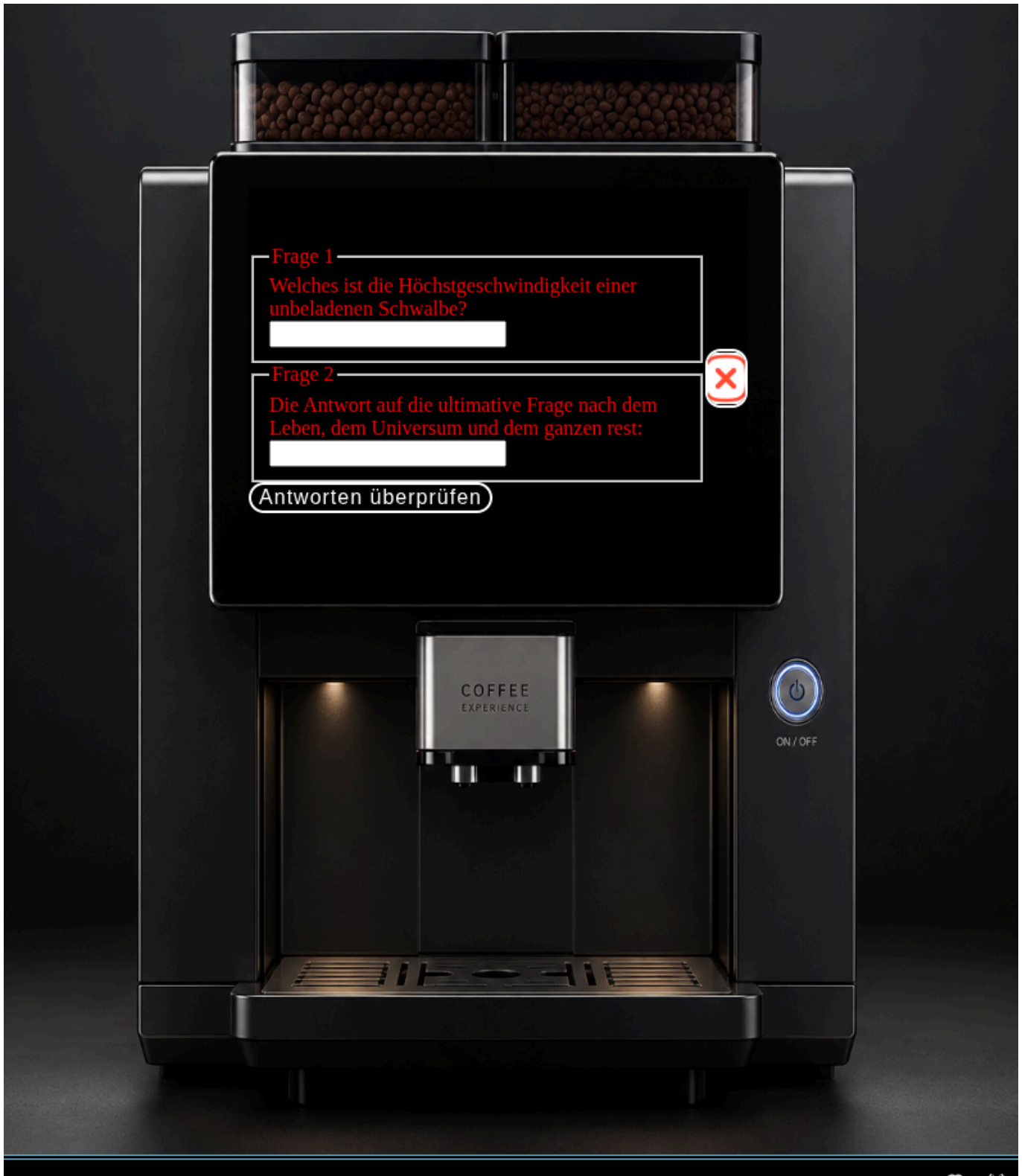


Abbildung: Fertiges Geheimmenü

Testen, Fehlersuche und Finetuning

Nachdem ich alle Menüs erstellt und alle Funktionen eingefügt habe, habe ich getestet, ob das Endergebnis meiner Vorstellung entsprach. Dabei sind dann hier und da kleine Fehler aufgekommen, z.B. dass bestimmte Elemente nicht versteckt wurden, wenn die Menüs gewechselt wurden.

Auch ist mir aufgefallen, dass die Fortschrittsanzeigen beim Getränke zubereiten/Reinigen weiterliefen, wenn der An/Aus Schalter betätigt wurden:



Abbildung: Funktion läuft bei ausgeschalteter Maschine trotzdem weiter

Die Lösung hierfür war eine einfache if-Abfrage vor dem Ausführend:

```
if(!power) return;
```

Dadurch bricht der Vorgang ab, sobald die Maschine "aus" ist.

Ich habe mit der Anwendung herumgespielt und mir den Code angeschaut. Dabei sind mir ein paar Redundanzen aufgefallen, welche ich beseitigt habe, indem ich Code in Funktionen zusammengefasst habe.

So war z.B. zu jedem Event Listener der Elemente es Spezialmenüs folgender Code zugeordnet:

```
alert("Nicht während der Arbeitszeit!");  
hauptMenü();
```

Daraus habe ich dann eine extra Funktion

```
function warnung(){  
    alert("Nicht während der Arbeitszeit!");  
    hauptMenü();  
}
```

gemacht, welche bei Klick einfach abgerufen wird. Ein kleiner Stolperstein war hierbei die Übergabe im Event-Listener: Ursprünglich hatte ich

```
btnIceT.addEventListener("click", warnung());
```

geschrieben. Durch die Klammern "()" wurde die Funktion jedoch sofort beim Laden der Seite ausgeführt statt erst beim Klick. Nach dem Entfernen der Klammern wird die Funktion nun korrekt als Referenz übergeben und erst beim Klick getriggert.

In dem Menü **Auffüllen** wird eine Statistik der Betriebsstoffe wie Kaffeebohnen, Wasser etc. angezeigt:



Abbildung: Auffüllmenü

Es gibt Variablen für jede Zutat, jedes ausgegebene Getränk verringert die Stände in den Variablen. Eine Funktion erstellt ein Array aus den aktuellen Werten und stellt diese dar. Beim Öffnen des Menüs wird die Funktion aufgerufen

```
//Füllstände anzeigen
function füllstandskontrolle(){
  const stoffe = ["Kaffeebohnen", "Brühe", "Kakaopulver", "Wasser(1)",
"Milch(1)"];
  const stände = [bohnen, brühe, kakao, wasser, milch];

  let ausgabe = "";

  for(let i = 0; i < stoffe.length; i++){
    ausgabe += `${stoffe[i]}: ${stände[i]}\n`;
  }
  füllStatistik.innerText = ausgabe;
}
```

In dem Untermenü lässt sich jede Substanz einzeln auffüllen, oder direkt alle auf einmal. Zusätzlich gibt es die Möglichkeit des Abbruches, welche direkt wieder ins Hauptmenü führt. Beim Testen der Anwendung war mir das etwas zu stark, ich wollte eine Möglichkeit haben, von dem Untermenü wieder zurück in das Obermenü zu wechseln. Also habe ich zusätzlich einen "Zurück" - Knopf implementiert:



Abbildung: Untermenü

Zuerst hatte ich den Code so geschrieben, dass ein Klick auf das Element das Untermenü versteckt und das Obermenü einfach wieder sichtbar macht. Das hatte zur Folge, dass die Füllstände nicht aktualisiert wurden, da die Funktion dafür nicht erneut aufgerufen wurde. Also habe ich meinen Coe angepasst:

```
//Zurück-Knopf
```

```
btnZurück.addEventListener("click", function(){
    auffüllenUntermenü.classList.add("versteckt");
    auswahl = "auffüllen";
    zeigeMenü();
    btnZurück.classList.add("versteckt");
});
```

Die zeigeMenü Funktion bestimmt anhand der Variable auswahl welches Menü "geladen" wird. Da es nur dieses eine Untermenü gibt, setzt der Klick die Variable auf das Obermenü und ruft die zeigeMenü Funktion auf, welche dann die Statistikfunktion des Auffüllen-Menüs neu aufruft, wodurch die Stände dann aktualisiert werden.

Da von den flüssigen Zutaten je Durchgang 0,3 abgezogen wurden, hat sich folgender Fehler bei der Ausgabe eingeschlichen:



Abbildung: Fehler in der Ausgabe

Bei 1.4 - 0.3 ist das Ergebnis 1.0999999999999999 statt 1.1.

Ich habe die Funktion der Füllstandsanzeige so umgeschrieben, dass die Stände von Wasser und Milch auf genau 1 Nachkommastelle präzisiert werden.

```
//Füllstände anzeigen
function füllstandskontrolle(){
    const stoffe = ["Kaffeebohnen", "Brühe", "Kakaopulver",
"Wasser(1)", "Milch(1)"];
    const stände = [bohnen, brühe, kakao, wasser.toPrecision(2),
milch.toPrecision(2)];

    let ausgabe = "";

    for(let i = 0; i < stoffe.length; i++){
        ausgabe += `${stoffe[i]}: ${stände[i]}\n`;
    }
    füllStatistik.innerText = ausgabe;
}
```

Das Geheimmenü war offen zugänglich, damit es seinem Namen gerecht wird, habe ich es versteckt. Da das Menü nun auch während eines Zubereitungsvoorganges aufrufbar war, habe ich einen Check geschrieben, der dies verhindert.

Ich habe die Variable

```
let zubereitungAktiv = false;
```

angelegt. Sobald die Fortschrittsanzeigefunktion aufgerufen wird, setzt diese den Wert auf **true** und nach Abschluss wieder zurück auf **false**. Die Funktion des Event Listeners für den Geheimmenü-button prüft nun sowohl den As/Aus Zustand wie auch die aktive Zubereitung über die entsprechenden Variablen.

```
btnGeheim.addEventListener("click", function(){
    if(!power || zubereitungAktiv) return;
    auswahl="geheim";
    zeigeMenü();
});
```

Das Menü kann nun nicht aufgerufen werden, wenn die Maschine aus ist (!power) aber auch nicht, falls ein Getränk in Zubereitung ist (zubereitunAktiv = true).

Ich habe mir von der KI ein Bild einer Kaffetasse erstellen lassen. Auch dafür habe ich in meinem HTML-Code ein "div" erstellt in welchem das Bild eingebettet ist und dieses über die Methode mit dem roten Rahmen an die entsprechende Position gebracht und die Größenverhältnisse angepasst. Das Bild hatte einen weißen Hintergrund, ich wollte aber einen transparenten haben, um nur die Tasse anzuzeigen. Nach einigem Ausprobieren und recherchieren habe ich das Bild auf mein iPhone transferiert. Dort ließ sich das Objekt sehr einfach aus dem Bild lösen und als PNG speichern, welches ich dann entsprechend meiner Vorstellungen verwenden konnte.

Die Fortschrittsfunktion für Getränke habe ich so erweitert, dass dieses Element beim Aufruf der Funktion nun eingeblendet wird, um die Illusion abzurunden. Am Ende ist ein Klick auf die Tasse erforderlich, um diese zu "entnehmen" und zum Hauptmenü zurückzukehren.

```
const kaffeeTasse = document.getElementById("tasse");

kaffeeTasse.addEventListener("click", function(){
  if(!zubereitungAktiv){
    kaffeeTasse.classList.add("versteckt");
    fortschritt.innerText = "";
    hauptMenü();
    zubereitungAktiv = false;
    return true;
  }
});
```

Für den letzten Feinschliff habe ich meinen CSS code an diversen Stellen angepasst, um Elemente entsprechend meiner Vorstellungen besser zu positionieren und zu zentrieren und die Größen anzupassen.

Dadurch, dass die die Fortschrittsanzeigen der Reinigung und der Getränkezubereitung auf das selbe HTML Element zu Anzeigezwecken zugegriffen haben, kam es zu Darstellungsfehlern/Überlagerungen, wenn man eines durch Betätigen des An-Aus-Schalters abgebrochen und dann das Andere aufgerufen hat. Ich habe daher ein zusätzliches HTML Element erstellt und die die Funktionen in JS jeweils so angepasst, dass sich jede Funktion eines anderen Elementes bedient.

Ich habe zum Abschluss (und periodisch nach Änderungen) meinen Code von der KI analysieren lassen, um mich auf Fehler welche mir entgangen waren hinweisen zu lassen. Auch zu Fragen bezüglich Syntax o-Ä. habe ich diese als Werkzeug hinzugezogen.

Abschließendes Fazit

Ich habe die Funktionalität so hergestellt, wie ich es mir vorgestellt habe. Allerdings hat der gesamte Vorgang länger gedauert, als ursprünglich angenommen. Ich kam daurch zeitlich etwas in den Verzug, sodass ich einige Funktionen aus dem Vorgangsprojekt, welche für mich nebensächlich waren, ausgesetzt wurden. Gerade im Bereich CSS habe dich doch häufiger recherchieren oder die KI fragen müssen, was den Vorgang verlangsamt hat.

?